

3주차 학습 자료

서버 환경 세팅과 앱 화면 내비게이션

FRONTEND · FLUTTER

화면 전환과 사용자 인터랙션

GoRouter · go/push/pop · Path/Query/Extra · ListView/GridView · ShellRoute + NavigationBar · GestureDetector · flutter_rating_bar · Dialog · BottomSheet · Snackbar

BACKEND · SPRING BOOT

첫 API 만들고 검증하기

INSERT · 3계층 아키텍처 · 커넥션 풀 · JdbcTemplate · 파라미터 바인딩 & SQL Injection · 환경변수 분리 · Postman 검증 · 생 SQL & No DTO의 한계

차례

1. 한 장 요약 — 이번 주 꼭 가져갈 것
2. FE 1. 화면 전환과 Route · GoRouter 설정
3. FE 2. go / push / pushReplacement / pop
4. FE 3. Path · Query · Extra, Mock Data
5. FE 4. ListView · GridView
6. FE 5. NavigationBar · ShellRoute vs StatefulShellRoute
7. FE 6. GestureDetector · flutter_rating_bar
8. FE 7. Dialog · BottomSheet · Snackbar
9. FE 8. 자주 발생하는 오류 · 내 구현 지도 · 셀프 체크
10. BE 1. INSERT의 뼈대와 3가지 실수
11. BE 2. 3계층 아키텍처 · 커넥션 풀
12. BE 3. 실습 1 GET /books · 실습 2 POST /books
13. BE 4. 환경변수 · Postman · 생 SQL의 대환장 파티
14. BE 5. 미션 3개 · 핵심 키워드 · 셀프 체크
15. FE ↔ BE 연결해서 보기

워크북 본문·실습·미션·체크리스트를 그대로 따라가며 정리했고, 코드는 내 저장소(movielog PR #5, umc-backend-study)에서 실제로 돌린 것만 실었다.

한 장 요약 — 이번 주 꼭 가져갈 것

Frontend 7문장

1. **Navigator**는 화면을 Stack에 직접 push/pop, **Router**는 URL로 화면 구성을 선언. `go_router`는 Router API를 편하게 쓰는 Flutter 팀 패키지.
2. 앱 최상위는 `MaterialApp.router(routerConfig: AppRouter.router)` 하나. 첫 화면은 `home`이 아니라 `initialLocation`이 정한다.
3. `go`는 위치를 바꾸고(Stack 안 쌓임), `push`는 위에 쌓는다. 탭 전환·로그인 완료는 `go`, 상세 보기는 `push`, 돌아오기는 `pop`.
4. **Path**는 리소스 식별(`/movies/:movieId`), **Query**는 선택 조건(`?genre=`), **Extra**는 객체 전달이지만 URL에 안 남아 재시작·Deep Link에서 사라진다.
5. **ShellRoute**는 `NavigationBar`를 한 번만 만들고 `child`만 바꾼다. 탭별 Stack까지 보존하려면 `StatefulShellRoute.indexedStack`.
6. 입력은 `RatingBar.builder` (0.5 단위), 표시는 `RatingBarIndicator` (4.3 같은 값 OK). 상태는 부모가 소유하고 자식은 `onChanged`로 알린다.
7. Dialog·BottomSheet의 선택값은 `Navigator.pop(context, value)`로 돌아온다. `Snackbar`는 `ScaffoldMessenger.of(context)`.

Backend 7문장

1. **INSERT**는 컬럼 순서 = VALUES 순서. `AUTO_INCREMENT` PK는 적지 않고, `NOT NULL` 컬럼은 빠지면 에러, FK는 부모가 먼저.
2. **Controller**(웨이터: 요청·응답) → **Service**(셰프: 비즈니스 규칙) → **Repository**(창고지기: SQL). 나누면 유지보수·재사용이 쉬워진다.
3. 연결을 매번 맺지 않고 **커넥션 풀**에서 빌려 쓴다. Spring Boot는 `application.yml`만 있으면 `HikariCP` + `JdbcTemplate`을 알아서 만들어 둔다.
4. 조회는 `queryForList`, 변경(INSERT/UPDATE/DELETE)은 `update`. 값은 문자열로 붙이지 말고 **? 파라미터 바인딩** — SQL Injection 방어.
5. DB 비밀번호는 코드에 쓰지 않는다. `${DB_PASSWORD:}` 환경변수 + `.env` (gitignore).
6. Postman으로 요청(URL/Body) → 상태 코드(200/201) → 응답 JSON을 눈으로 확인하고, GET으로 재검증한다.
7. 생 SQL + No DTO는 **오타가 런타임까지 가고, snake_case가 프론트로 새고, ? 순서 맞추기**가 지옥 — 4주차 DTO·ORM의 이유.

이번 주 실제로 만든 것

movielog (PR #5) — Route 6개(`/start` `/register` `/home` `/movies?genre=` `/my` `/movies/:movieId`), 화면 4개, 공통 위젯 9개, 테스트 12개

umc-backend-study — Spring Boot 4.1.1 · Java 21 · API 5개 (`GET/POST /books` , `GET /books/category/{id}` , `POST /rentals` , `PATCH /rentals/{id}/return`)

화면 전환과 Route · GoRouter 설정

1. 세 가지 방식

이름	설명	언제
Navigator	Route를 Stack처럼 직접 push·pop	작은 앱의 단순 전환
Router	URL과 앱 상태를 기준으로 화면 구성을 선언	URL·Deep Link·여러 Navigator
go_router	Flutter Router API를 편하게 쓰는 Flutter 팀 패키지 (flutter pub add go_router)	이번 주부터 기본

2. AppRouter 클래스 — 라우팅 코드는 별도 파일로

```
// lib/router/app_router.dart
class AppRouter {
  AppRouter._(); // private 생성자: 외부에서 new 못 함

  // 앱 전체에서 라우터는 하나 → static final
  static final router = GoRouter(
    initialLocation: '/start', // 첫 화면
    routes: [
      GoRoute(
        path: '/start', // URL 패턴
        builder: (context, state) => const StartScreen(),
      ),
      GoRoute(
        path: '/register',
        builder: (context, state) => const SignUpScreen(),
      ),
    ],
  );
}
```

GoRouter

initialLocation 기본 시작 위치 ·
routes Route 목록

GoRoute

path URL 경로 패턴 · builder 그
경로에서 표시할 Widget

AppRouter._()

외부에서 객체 생성을 막는 private 생성
자

static final

앱 전체가 라우터 하나를 공유

새 화면이 생기면 MaterialApp이 아니라
AppRouter.routes 에 GoRoute 를 추가한다.

3. MaterialApp → MaterialApp.router

```
// 기존
return MaterialApp(
  title: 'MovieLog',
  theme: AppTheme.light,
  home: const StartScreen(), // 첫 화면을 직접 지정
);
```

```
// 변경
return MaterialApp.router(
  title: 'MovieLog',
  theme: AppTheme.light,
  routerConfig: AppRouter.router, // 첫 화면·이동은 Router가
);
```

주의 MaterialApp 과 MaterialApp.router 를 함께 쓰지 않는다. 최상위에는 MaterialApp.router 하나만 두고, 첫 화면은 initialLocation 이 결정한다.

go / push / pushReplacement / pop

차이는 이동 후 기존 화면을 어떻게 처리하는가다.

context.push('/detail')

[Home] → [Detail] ← 현재

이전 화면 유지, 위에 추가. 상세 보기(상품·게시글·프로필).

pushReplacement('/home')

[Login] → [Home]

현재 화면 제거 후 교체. 뒤로 가도 Login으로 안 돌아감. 로그인·회원가입·온보딩 완료.

context.pop()

[Home] ← 현재
[Detail] →

현재 화면 제거, 이전 화면으로. AppBar 뒤로 가기 버튼.

context.go('/home') — Stack에 쌓는 게 아니라 URL(Route 위치) 자체를 변경

/login ↓ go('/home') /home

로그인 완료 후 홈, BottomNavigationBar 탭 전환처럼 현재 위치를 새 경로로 바꿀 때.

한눈에 비교

메서드	기존 화면	새 화면	뒤로가기	언제
go()	새 Route 구조로 변경	이동	Route 구조에 따라 다름	탭 전환, 로그인 완료
push()	유지	Stack에 추가	가능	상세 보기
pushReplacement()	제거	현재 화면과 교체	이전으로 못 돌아감	로그인·온보딩 완료
pop()	현재 화면 제거	이전 화면 표시	이전 화면으로	뒤로 가기

내 코드에서

위치	메서드	이유
시작하기 버튼	go('/register')	시작 화면으로 돌아올 필요 없음
회원가입 _submit()	go('/home')	회원가입이 Stack에 남으면 안 됨
NavigationBar	go('/home' '/movies' '/my')	push면 탭마다 Stack이 쌓임
영화 카드 onTap	push('/movies/\${movie.id}')	목록으로 돌아와야 함
상세 _goBack()	canPop() ? pop() : go('/home')	URL 직접 진입이면 Stack이 비어 있음

회원가입·홈에서 뒤로 가기 금지 (미션 요구)

go로만 이동해 Stack에 이전 화면이 없고, 추가로 PopScope(canPop: false) 로 감싸 Android 뒤로 가기도 막았다.

테스트에서 확인한 사실

routerDelegate.currentConfiguration.uri 는 go로 정한 기준 위치만 돌려주고 push된 화면은 반영하지 않는다. 맨 위 화면은 router.state.uri. "go는 위치를 바꾸고 push는 쌓는다"가 API에도 그대로 드러난다.

Path Parameter · Query Parameter · Extra, Mock Data

	Path Parameter	Query Parameter	Extra
용도	리소스 식별. Route에 꼭 포함(영화 ID)	검색어·필터·정렬 같은 선택적 조건	이미 가진 객체를 편하게 전달
보내기	<code>context.push('/movies/\${movie.id}')</code>	<code>Uri(path: '/movies', queryParameters: {'genre': 'drama'}).toString() → go</code>	<code>context.push('/movies/1', extra: movie)</code>
읽기	<code>state.pathParameters['movieId']</code>	<code>state.uri.queryParameters['genre']</code>	<code>state.extra as Movie?</code>
URL에	남음 <code>/movies/1</code>	남음 <code>/movies?genre=SF</code>	안 남음
재시작·Deep Link	살아 있음	살아 있음	null

Extra만 의존하면 앱 재시작·URL 직접 접근·Deep Link에서 값이 없다. 그래서 Required Mission은 **Path Parameter의 ID로 Mock Data를 다시 찾는** 방식을 기본으로 한다.

Route 선언과 읽기

```
GoRoute(
  path: '/movies/:movieId', // ':이름'
  builder: (context, state) => MovieDetailScreen(
    // Key는 ':movieId'와 똑같이. 숫자 아니면 null
    movieId: int.tryParse(
      state.pathParameters['movieId'] ?? ''),
  ),
),
```

Query로 상태를 URL이 소유하기 (Challenge)

```
void _applyGenres(BuildContext context,
Set<String> sel) {
  final location = Uri(
    path: '/movies',
    queryParameters: sel.isEmpty
      ? null // 없으면
      : {'genre': sel.join(',')}, // ?genre=드라마,SF
  ).toString();
  context.go(location); // 같은 화면이 새 값으로 다시 그려짐
}
```

→ `MovieListScreen` 이 `StatelessWidget`이 된다. `setState` 없이 URL이 상태.

Mock Movie 데이터 — 홈·목록·상세가 같은 것을 읽는다

```
class Movie {
  const Movie({required this.id, required this.title,
    required this.genre, required this.year,
    required this.posterAsset, this.rating});
  final int id; final String title; final String genre;
  final int year; final String posterAsset;
  final double? rating;
}

const movies = [
  Movie(id: 1, title: '별빛 아래 우리', genre: '드라마',
    year: 2024, posterAsset: 'assets/.../hero.jpg',
    rating: 4.5),
  // ... 6편
];

Movie? findMovieById(int? id) {
  for (final movie in movies) {
    if (movie.id == id) return movie;
  }
  return null; // 없는 ID → null → "찾을 수 없어요"
}
```

왜 Movie? (nullable)인가

URL을 직접 치거나 잘못된 ID로 들어올 수 있다. null이면 상세 화면이 "없음"을 그리고, !를 쓰면 앱이 죽는다. '별빛 아래 우리'는 홈·목록·상세에서 같은 ID(1)·제목을 쓴다.

ListView · GridView — 반복 레이아웃

Widget	사용하는 경우	핵심 속성
<code>ListView.builder</code>	항목을 한 방향으로 나열 + 스크롤 (세로 기본, 가로 가능)	<code>itemCount</code> 개수 · <code>itemBuilder</code> index로 Widget 생성 · <code>scrollDirection</code> <code>Axis.vertical</code> / <code>horizontal</code> · <code>padding</code>
<code>ListView.separated</code>	항목 사이에 간격·구분선	+ <code>separatorBuilder</code> (항목과 항목 사이에만 호출됨)
<code>GridView.builder</code>	행·열 격자. 한 줄에 2개 이상 (포스터·갤러리·상품 카드)	<code>itemCount</code> · <code>itemBuilder</code> · <code>gridDelegate</code>

가로 ListView.separated (홈 영화 줄)

```

    SizedBox(
      height: 236, // 가로 ListView는 높이를 스스로 못 정함
      child: ListView.separated(
        scrollDirection: Axis.horizontal,
        padding: const
      EdgeInsets.symmetric(horizontal: 16),
        itemCount: movies.length,
        separatorBuilder: (context, index) =>
          const SizedBox(width: 12),
        itemBuilder: (context, index) {
          final movie = movies[index]; // List에서 하나
          // 꺼냄
          return MovieCard(movie: movie, width: 136,
            onTap: () => onMovieTap(movie));
        },
      ),
    )
  )

```

GridView.builder 2열 (영화 목록)

```

    GridView.builder(
      padding: const EdgeInsets.fromLTRB(16, 0, 16, 24),
      gridDelegate:
        const
        SliverGridDelegateWithFixedCrossAxisCount(
          crossAxisCount: 2, // 한 줄에 2개
          crossAxisSpacing: 12, // 좌우 간격
          mainAxisSpacing: 16, // 위아래 간격
          childAspectRatio: 0.6, // 가로/세로 (포스터 2:3 +
            글자 2줄)
        ),
      itemCount: filtered.length,
      itemBuilder: (context, index) {
        final movie = filtered[index];
        return MovieCard(movie: movie,
          onTap: () =>
            context.push('/movies/${movie.id}'));
      },
    )
  )

```

<code>crossAxisCount</code>	한 줄에 표시할 항목 개수
<code>crossAxisSpacing</code>	좌우 항목 사이 간격
<code>mainAxisSpacing</code>	위아래 항목 사이 간격
<code>childAspectRatio</code>	각 항목의 가로 / 세로 비율

같은 **MovieCard** 를 두 곳에서 쓰는 요령 — 가로 ListView에서는 `width` 를 고정하고, GridView에서는 `width: null` 로 셀 너비를 따른다. 포스터는 `Expanded` 로 남는 세로 공간을 채우고, 글자 두 줄은 고정 높이. 부모가 높이를 정해 주는 자리에서만 쓰는 전제.

NavigationBar · ShellRoute vs StatefulShellRoute.indexedStack

MainScreen — 탭 화면을 담는 공통 부모

NavigationBar는 홈·영화·마이에 공통, 각 탭 화면은 다르다. 그래서 Scaffold와 NavigationBar를 한 번만 관리하는 부모를 만든다.

```
class MainScreen extends StatelessWidget {
  const MainScreen({super.key,
    required this.currentIndex, required
    this.child});
  final int currentIndex; // URL에서 계산한 탭 번호
  final Widget child; // ShellRoute가 넘겨준 탭
  화면

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: child,
      bottomNavigationBar: NavigationBar(
        selectedIndex: currentIndex,
        onDestinationSelected: (index) {
          switch (index) {
            case 0: context.go('/home');
            case 1: context.go('/movies');
            case 2: context.go('/my');
          }
        },
        destinations: const [
          NavigationDestination(icon:
            Icon(Icons.home_outlined),
            selectedIcon: Icon(Icons.home), label:
            '홈'),
          NavigationDestination(icon:
            Icon(Icons.movie_outlined),
            selectedIcon: Icon(Icons.movie),
            label: '영화'),
          NavigationDestination(icon:
            Icon(Icons.person_outline),
            selectedIcon: Icon(Icons.person),
            label: '마이'),
        ],
      ),
    );
  }
}
```

selectedIndex 현재 선택된 탭
onDestinationSelected 탭을 눌렀을 때 실행
destinations NavigationDestination 목록

GoRouter — ShellRoute

```
ShellRoute(
  builder: (context, state, child) {
    return MainScreen(
      // Query가 붙어도 path만 보므로 탭 계산이 안 흔들림
      currentIndex:
        indexFromLocation(state.uri.path),
      child: child,
    );
  },
  routes: [
    GoRoute(path: '/home', builder: ...
      HomeScreen()),
    GoRoute(path: '/movies', builder: ...
      MovieListScreen(...)),
    GoRoute(path: '/my', builder: ...
      MyPageScreen()),
  ],
),

static int indexFromLocation(String path) {
  if (path.startsWith('/movies')) return 1;
  if (path.startsWith('/my')) return 2;
  return 0;
}
```

핵심 child만 바뀌므로 NavigationBar가 탭마다 중복 생성되지 않고, URL에서 계산한 currentIndex로 선택 상태와 화면이 항상 일치한다.

StatefulShellRoute.indexedStack

— 탭별 상태 보존

	ShellRoute	StatefulShellRoute.indexedStack
탭 전환	context .go()	navigationShell.goBranch(index)
탭별 Stack	안 남음	StatefulShellBranch마다 독립 Navigator, 스크롤·입력·상세 화면 유지
메모리	가벼움	모든 Branch가 IndexedStack에 유지
쓸 때	단순 최상위 탭 이동	영화 탭에서 상세로 갔다가 홈 갔다 와도 상세가 그대로여야 할 때

내 선택: 상세를 Shell 밖(전체 화면, NavigationBar 없음)에 두어 탭별 Stack 보존이 필요 없었으므로 ShellRoute. 4주차 이후 탭 안에 하위 화면이 생기면 전환 검토.

GestureDetector · flutter_rating_bar

GestureDetector — Widget에 제스처 연결

<code>child</code>	제스처를 감지할 Widget
<code>onTap</code>	한 번 눌렀을 때
<code>onDoubleTap</code>	두 번
<code>onLongPress</code>	길게
<code>behavior</code>	투명한 영역의 Touch 처리. HitTestBehavior.opaque 면 빈 여백을 눌러도 Tap

```
GestureDetector(
  behavior: HitTestBehavior.opaque,
  onTap: () =>
    context.push('/movies/${movie.id}'),
  child: MovieCardContent(movie: movie),
)
```

Material 물결 효과가 필요하면 같은 역할의 `InkWell`. 3주차는 `GestureDetector`의 `onTap` 으로 카드 → 상세 Route.

부모가 정한다 — `MovieCard` 는 `onTap`:
`VoidCallback` 만 받고, 어디로 갈지는 홈·목록·마이페이지
가 각자 넘긴다.

flutter_rating_bar (flutter pub add flutter_rating_bar)

Widget	용도	별점 단위
<code>RatingBar</code> <code>.builder</code>	사용자 가 입력	0.5, 1.0, 1.5 ... (<code>minRating:</code> <code>0.5</code> , <code>allowHalfRating:</code> <code>true</code>)
<code>RatingBar</code> <code>Indicator</code>	읽기 전 용 표시	3.1, 3.7, 4.3 같은 소수도 그대로

```
// 입력 위젯: 어떻게 입력할지만 담당, 값은 부모가 소유
class MovieRatingInput extends StatelessWidget {
  const MovieRatingInput({super.key,
    required this.rating, required
    this.onChanged});
  final double rating;
  final ValueChanged<double> onChanged;

  @override
  Widget build(BuildContext context) =>
    RatingBar.builder(
      initialRating: rating,
      minRating: 0.5, allowHalfRating: true,
      itemCount: 5, itemSize: 40,
      itemBuilder: (c, i) => const Icon(Icons.star,
        color: Colors.amber),
      onRatingUpdate: onChanged, // 새 값 → 부모
    );
}

// 부모에서
double rating = 3.0;
MovieRatingInput(rating: rating,
  onChanged: (value) => setState(() => rating =
    value));

// 읽기 전용 (평균 평점 4.5)
Row(children: [
  RatingBarIndicator(rating: 4.5, itemCount: 5,
    itemSize: 24,
    itemBuilder: (c, i) => const Icon(Icons.star,
      color: Colors.amber)),
  const SizedBox(width: 8), const Text('4.5'),
])
```

별을 눌러도 숫자가 안 바뀌면 `onRatingUpdate` 로 받은 값을 State 변수에 저장하고 `setState` 를 호출했는지 확인.

Dialog · BottomSheet · Snackbar

Dialog — 확인이 필요한 상황(삭제·확정)

showDialog context · builder · barrierDismissible 바깥 눌러 닫기 여부

AlertDialog icon title content actions backgroundColor shape scrollable

Dialog 내부 UI 자유(커스텀). Column(mainAxisSize: MainAxisSize.min)

```
// 커스텀 Dialog (StatefulWidget) - 값 돌려주기
ElevatedButton(
  onPressed: hasRating
    ? () => Navigator.pop(context, _rating) // 2번째 인자가 결과
    : null, // 별 없으면 비활성
  child: const Text('확인'),
)

// 띄우기 - pop으로 넘긴 값이 반환값
final rating = await showDialog<double>(
  context: context,
  builder: (dialogContext) =>
    RatingDialog(initialRating: _myRating ?? 0),
);
if (rating == null || !mounted) return; // 바깥 눌러 닫으면 null
setState(() => _myRating = rating);
```

Snackbar · ScaffoldMessenger — 짧은 결과 안내

content 안내 Widget

duration 표시 시간

behavior fixed / floating

action 선택적 추가 행동

```
ScaffoldMessenger.of(context)
  ..hideCurrentSnackBar() // 연타 시 안 쌓이게
  ..showSnackBar(SnackBar(
    content: Text(_isFavorite
      ? '즐거찾기에 추가했습니다.' : '즐거찾기에서 삭제했습니다.'),
    behavior: SnackBarBehavior.floating,
    duration: const Duration(seconds: 2),
  ));
```

BottomSheet — 화면을 벗어 나지 않는 추가 작업(필터·정렬)

showModalBottomSheet context · builder · isScrollControlled 큰 높이 허용 · useSafeArea 위·좌·우 시스템 영역 회피

useSafeArea false (기본) 짧은 선택 Sheet / true 긴 목록·위쪽까지 커지는 Sheet

```
final result = await
showModalBottomSheet<Set<String>>(
  context: context,
  isScrollControlled: true, // Draggable로 높이 조절하려면 true
  useSafeArea: true,
  builder: (sheetContext) => GenreFilterSheet(
    genres: genres, initialSelected:
selectedGenres),
);
if (result == null || !context.mounted) return;
_applyGenres(context, result);
```

DraggableScrollableSheet — 드래그로 높이 조절

```
DraggableScrollableSheet(
  expand: false, // 부모 전체를 강제로 채우지 않음
  initialChildSize: 0.5, // 처음 높이 비율
  minChildSize: 0.3, maxChildSize: 0.9,
  builder: (context, scrollController) =>
    Column(children: [
      Expanded( // 목록만 스크롤
        child: ListView.builder(
          controller: scrollController, // 드래그 ↔ 스크롤 연결 필수
          itemCount: genres.length,
          itemBuilder: (c, i) => CheckboxListTile(
            value: _selected.contains(genres[i]),
            onChanged: (v) => _toggle(genres[i], v),
            title: Text(genres[i])),
        ),
      ),
      Padding(padding: const EdgeInsets.all(16), // 확인 버튼 고정
        child: SizedBox(width: double.infinity,
          child: ElevatedButton(
            onPressed: () => Navigator.pop(context, _selected),
            child: Text(_selected.isEmpty ? '전체 영화 보기'
              : '${_selected.length}개 장르 적용'))),
    ]),
)
```

Expanded가 핵심 — 목록만 스크롤, 버튼은 고정. **확인을 눌러야** 목록에 반영.

자주 발생하는 오류 · 내 구현 지도 · 셀프 체크

자주 발생하는 오류 (워크북)

증상	확인
MaterialApp과 MaterialApp.router를 함께 씀	최상위는 MaterialApp.router(routerConfig:) 하나
뒤로 가기가 예상과 다름	상세 이동에 go/push 중 뭘 썼나. 목록으로 돌아와야 하면 push
Path Parameter가 null	:movieId 이름과 pathParameters['movieId'] Key 동일인가
BottomSheet 뒤에서 Snackbar 안 보임	Sheet를 먼저 닫은 뒤 현재 Scaffold의 ScaffoldMessenger로 표시
별 눌러도 숫자 안 바뀜	onRatingUpdate 값 저장 + setState

내가 겪은 것 4건

1. MaterialApp(home:) 으로 띄운 테스트에서 context.go → No GoRouter found. 테스트도 MaterialApp.router 로.
2. push한 상세가 떠 있는데 currentConfiguration.uri 는 /home → 맨 위 화면은 router.state.uri .
3. Sheet 화면 밖 Checkbox를 누르니 그 좌표의 확인 버튼이 눌림 → 드래그로 펼친 뒤 선택.
4. Snackbar 2초를 흘려도 버튼을 가림 → 등장 애니메이션이 끝나야 타이머 시작.

셀프 체크 — 최종 체크리스트를 질문으로

- ☐ MaterialApp.router와 GoRouter의 관계는?
routerConfig에 GoRouter 하나를 넘기고, 첫 화면·이동은 GoRouter(initialLocation, routes)가 담당한다.
- ☐ go와 push의 차이를 한 문장으로?
go는 현재 Route 위치(URL)를 바꾸고, push는 현재 화면 위에 새 화면을 쌓는다.
- ☐ 로그인 완료 후 홈으로 갈 때 push를 쓰면 무슨 문제?
뒤로 가기로 로그인 화면이 다시 나온다. go 또는 pushReplacement.
- ☐ Path / Query / Extra 중 앱 재시작 후 사라지는 것은?
Extra. URL에 남지 않기 때문.
- ☐ NavigationBar 선택 상태와 화면을 일치시키는 방법은?
state.uri.path로 selectedIndex를 계산해 MainScreen에 넘긴다.
- ☐ Dialog에서 고른 값은 어떻게 부모로 돌아오나?
Navigator.pop(context, value)의 두 번째 인자가 showDialog의 반환값.
- ☐ BottomSheet에서 목록은 스크롤되고 확인 버튼은 고정되게 하려면?
Column 안에서 ListView만 Expanded에 넣고 버튼은 밖에 둔다.
- ☐ RatingBar.builder와 RatingBarIndicator를 언제 나눠 쓰나?
입력은 builder(0.5 단위), 저장된 값 표시는 Indicator(소수 그대로).

내 구현 지도 (movielog PR #5)

```
MaterialApp.router (AppRouter.router)
| /start      StartScreen
| /register   SignUpScreen (PopScope
canPop:false)
| ShellRoute MainScreen (Scaffold +
NavigationBar)
| | /home     HomeScreen
| |           FeaturedMovieCard · HorizontalMovieList
×2
| | /movies   MovieListScreen (Stateless, 장르는
URL Query)
| |           GenreChips · GridView 2열 · 필터 →
GenreFilterSheet
| |           ↳ /my      MyPageScreen (1주차 프로필 위젯 재사용)
| |           ↳ /movies/:movieId MovieDetailScreen (Shell 밖,
push/pop)
| |           MovieAverageRating(4.5) · 즐겨찾기→Snackbar
| |           · 평점 남기기 → RatingDialog → Snackbar
```

상태	소유자
선택 장르	URL Query
Sheet 안 체크	_GenreFilterSheetState (확인 전까지)
즐거찾기·내 평점	_MovieDetailScreenState
Dialog 별점	_RatingDialogState → pop으로 전달

INSERT의 뼈대와 초보자가 가장 많이 하는 실수 3가지

뼈대 — "어떤 테이블의 어떤 컬럼에, 어떤 값을"

```
INSERT INTO 테이블명 (컬럼1, 컬럼2, 컬럼3)
VALUES (값1, 값2, 값3);
```

INSERT INTO 데이터를 추가할 테이블
(컬럼 ...) 값을 채울 컬럼 이름을 순서대로
VALUES (...) 컬럼 순서와 1:1로 정확히 일치하는 실제 값

순서와 타입 매칭 규칙

- (title, category_id) 로 적었으면 값도 ('클린 코드', 1) — 순서·개수 일치
- 문자열(VARCHAR, TEXT)·날짜(DATETIME)는 작은따옴표, 숫자(INT)·불리언(TRUE/FALSE)은 따옴표 없이

2주차 02_seed.sql로 다시 보기 — FK 순서

```
-- 1. 부모: 사용자
INSERT INTO users (nickname) VALUES ('민서'), ('수현');
-- 2. 부모: 카테고리
INSERT INTO category (name) VALUES ('문학'), ('과학');
-- 3. 자식: 도서 (category_id FK)
INSERT INTO book (category_id, title, description, is_available)
VALUES (1, '달빛 도서관', '소설', TRUE);
-- 4. 손자: 대여 (user_id, book_id FK)
INSERT INTO rental (user_id, book_id, rented_at, due_at, returned_at)
VALUES (1, 1, '2026-08-10 10:00:00', '2026-08-17 10:00:00', NULL);
```

사용자·카테고리 → 책 → 대여. 1주차 ERD의 외래키 관계 순서 그대로.

실수 3가지

실수	규칙
1. PK(id) 적음	AUTO_INCREMENT 는 DB가 1, 2, 3... 알아서 붙인다. INSERT에 id 컬럼을 적지 않는다.
2. 필수 컬럼 누락	DEFAULT 가 있거나 NULL 허용이면 생략 가능(returned_at). 기본값 없는 NOT NULL (title)을 빼면 DB 에러.
3. FK 위반	없는 category_id 999 를 넣으면 DB 가 거부. 부모 데이터가 먼저 있어야 자식을 넣을 수 있다.

이번 주 실제로 만난 두 에러

"titel" 오타 → title이 null → Column 'title' cannot be null (실수 2)

categoryId: 999 → a foreign key constraint fails (실수 3)

미션에서 쓴 INSERT / UPDATE

```
-- 미션 2: 대여 생성. rented_at=지금, due_at=7일 뒤,
-- returned_at은 NULL 허용이라 생략 → NULL
INSERT INTO rental (user_id, book_id, rented_at, due_at)
VALUES (?, ?, NOW(), DATE_ADD(NOW(), INTERVAL 7 DAY));
```

```
-- 선택 미션: 반납. 이미 반납된 건은 건드리지 않음
UPDATE rental SET returned_at = NOW()
WHERE rental_id = ? AND returned_at IS NULL;
```

UPDATE·DELETE에서 WHERE 를 빼면 전체 행이 바뀐다.

3계층 아키텍처 · DB 연결 통로(커넥션 풀)

식당으로 이해하는 3계층 — 역할과 책임을 칼같이 분리

계층	비유	하는 일	내 코드
Controller	웨이터 / 홀 매니저	손님(클라이언트)의 주문(HTTP 요청)을 가장 먼저 맞이. 빈 값 체크 → 주방에 넘김 → 결과를 접시(JSON)에 담아 서빙(응답)	@RestController BookController, RentalController
Service	메인 셰프 / 주방장	핵심 요리 규칙(비즈니스 로직). "대여 한도 초과?", "이미 반납된 건?" 같은 정책 검증. 필요하면 창고 관리자를 부름	@Service BookService, RentalService(중복 반납 → 404)
Repository	식자재 창고 관리자	DB에 직접 SQL을 날리는 유일한 계층. SELECT로 꺼내고 INSERT로 넣음. 손님·규칙은 신경 안 씀	@Repository BookRepository, RentalRepository

왜 3개로 나누나

① 독립적인 유지보수 — 컬럼명이 바뀌면 Repository의 SQL만 고친다. ② 코드 재사용 — 도서 조회 Repository 하나를 '전체 조회', '추천 목록', '관리자 검수' 등 여러 Service가 재활용.

Controller 하나에 다 넣으면? 돌아가긴 한다... 스파게티 코드가 된다. 3계층은 "내일의 내가 오늘의 나를 원망하지 않게 만드는" 최소한의 안전장치.

"지금 Service가 Repository 호출만 하고 끝인데 필요한가?" → 단순해서 그렇게 보일 뿐, 나중에 규칙이 들어가는 곳이 Service다. 이번 주 `returnBook()`의 "갱신 0건이면 404"가 첫 규칙.

왜 매번 연결하지 않고 커넥션 풀을 쓸까

비효율 (매 요청마다)

요청 → 문 두드림 → 비밀번호 확인 → 연결 → 쿼리 → 끊음
요청 → 문 두드림 → 비밀번호 확인 → 연결 → 쿼리 → 끊음
... 연결 맺고 끊는 데 시간 다 쓰고 뺨음

커넥션 풀 (서버 켤 때 핫라인 10개를 바꾸니에)

요청 → [풀에서 하나 빌림] → 쿼리 → [돌려줌]
요청 → [풀에서 하나 빌림] → 쿼리 → [돌려줌]

Spring Boot — "이미 준비된 풀옵션 핫라인"

`application.yml`의 DB 정보를 읽어 서버가 켜질 때 HikariCP와 JdbcTemplate을 컨테이너에 알아서 만들어 둔다. 별도 설정 코드 없이 Repository에서 @RequiredArgsConstructor로 주입받아 쓰면 끝.

NestJS — "직접 조립하는 Provider"

ORM 없이 순수 드라이버(mysql2)를 쓰면

`database.provider.ts`에서

`mysql.createPool({connectionLimit: 10, waitForConnections: true, ...})`를 직접 선언하고

`app.module.ts`의 providers에 등록해야 한다. 4주차 ORM 도입 시 사라지는 임시 코드.

내가 본 것 — 서버 기동 로그에는 Hikari가 없었고, GET /books를 처음 호출할 때 HikariPool-1 - Starting... Added connection... Start completed가 찍혔다. 풀은 첫 쿼리 때 초기화(lazy).

실습 1 GET /books · 실습 2 POST /books

Workbench에서 손으로 `SELECT * FROM book;` 치던 것을, Postman → 서버 → DB → 응답으로 이어지는 **완제품 API**로. 만드는 순서는 밑 단부터: Repository → Service → Controller.

1단계 Repository — 창고지기에게 쿼리 시키기

```
@Repository          // "나 창고지기 부품이야" 등록
@RequiredArgsConstructor // final 필드 생성자 자동 생성
→ 주입
public class BookRepository {
    private final JdbcTemplate jdbcTemplate; // 스프링이 준비해 둔 DB 도구

    public List<Map<String, Object>> findAll() {
        String sql = "SELECT * FROM book";
        // 결과 한 행 = Map. Key 컬럼명(title), Value 값(달빛 도서관)
        return jdbcTemplate.queryForList(sql);
    }

    public void save(Map<String, Object> body) {
        // book_id는 AUTO_INCREMENT라 생략,
        is_available은 true
        String sql = "INSERT INTO book
            (category_id, title, "
            + "description, is_available) VALUES
            (?, ?, ?, true)";
        // update(): INSERT/UPDATE/DELETE. 뒤의 인자가 ? 순서대로
        jdbcTemplate.update(sql,
            body.get("categoryId"),
            body.get("title"),
            body.get("description"));
    }
}
```

2단계 Service — 셰프

```
@Service
@RequiredArgsConstructor
public class BookService {
    private final BookRepository bookRepository;
    public List<Map<String, Object>> getAllBooks() {
        return bookRepository.findAll(); // 지금은 가공 없음
    }

    public void createBook(Map<String, Object> body) {
        bookRepository.save(body);
    }
}
```

3단계 Controller — 카운터 & 라우팅

```
@RestController          // 반환값을 JSON 본문으로 서빙
@RequestMapping("/books") // 기본 주소
@RequiredArgsConstructor
public class BookController {
    private final BookService bookService;

    @GetMapping            // GET /books
    public List<Map<String, Object>> getBooks() {
        return bookService.getAllBooks();
    }

    @PostMapping           // POST /books
    public String createBook(@RequestBody
        Map<String, Object> body) {
        bookService.createBook(body); // JSON
        Body → Map
        return "도서 등록이 완료되었습니다!";
    }
}
```

어노테이션	뜻
@Repository @Service @RestController	스프링 컨테이너에 부품(Bean)으로 등록. RestController는 View가 아닌 JSON 응답
@RequiredArgsConstructor	(Lombok) final 필드를 받는 생성자 생성 → 생성자 주입
@RequestMapping("/books")	컨트롤러 기본 주소
@GetMapping @PostMapping @PatchMapping	HTTP 메서드 + 경로 → 실행할 메서드
@RequestBody	JSON Body를 자바 객체(Map)로
@PathVariable	URL의 {categoryId} 값을 파라미터로

SQL Injection — 문자열 더하기 금지

"INSERT ... VALUES (' + title + '');" 에 ');
DROP TABLE book; -- 가 들어오면 테이블이 날아간다. 값 자리에 ? 를 뚫어 두고 드라이버가 안전한 텍스트로 치환하게 하는 **파라미터 바인딩**을 반드시 쓴다.

jdbcTemplate.update(sql, a, b, c) 가 그것.

🐱 NestJS 대응: `pool.query(sql) → [rows]`,
`pool.execute(sql, [a, b, c])` 로 바인딩, 새 파일은 `app.module.ts` 에 등록.

환경변수 분리 · Postman 검증 · 생 SQL & No DTO의 대환장 파티

민감 정보는 코드에 하드코딩하지 않는다

```
# src/main/resources/application.yml
spring:
  config:
    import: optional:file:.env[.properties] #
.env를 읽음
  datasource:
    url:
jdbc:mysql://${DB_HOST:localhost}:${DB_PORT:3306}/${DB_NAME:book_rental}
    username: ${DB_USER:root}
    password: ${DB_PASSWORD:} # 기본값을 비워 코드
에 안 남김

# .env (.gitignore에 추가, .env.example만 커밋)
DB_HOST=localhost
DB_PASSWORD=
```

`${이름:기본값}` — 환경변수가 없으면 기본값. NestJS는 `ConfigModule.forRoot({isGlobal: true}) + configService.get('DB_HOST', 'localhost')`.

Postman으로 찢어보고 관찰하기

#	요청	기대 결과
1	GET localhost:8080/books	200, JSON 배열
2	POST /books Body(JSON) { "categoryId": 1, "title": "클린 코드", "description": "애자일 소프트웨어 장인 정신" }	200, "도서 등록이 완료되었습니다!"
3	다시 GET /books	맨 끝에 "클린 코드" — DB 검증

Spring은 8080, NestJS는 3000. Body 탭 → raw → JSON 선택 후 붙여넣기.

직접 겪어보는 3가지 문제

문제	체험
1. 오타 하나에 서버가 침묵 Type Safety 부재	"title": "해리포터" 로 보내면 컴파일러는 빨간 줄 하나 없다. body.get("title") 이 null → DB까지 내려가 Column 'title' cannot be null → 500. 클라이언트 입력 오류인데 서버 잘못처럼 보인다.
2. DB 스키마와 프론트의 강한 결합	GET /books 응답에 book_id, category_id, is_available — DB의 snake_case 컬럼명이 여과 없이 프론트로. 컬럼명을 바꾸면 화면이 와르르.
3. 쿼리 문자열 하드코딩 지옥	컬럼 20개면 ? 20개의 순서와 파라미터 20개의 순서를 눈으로 맞춰야 한다. 반드시 실수한다.

4주차 예고 — DTO와 ORM

DTO + @Valid (@NotBlank title)면 Controller 단계에서 400으로 걸러지고 필드명이 고정된다. ORM(JPA)이면 엔티티 필드 ↔ 컬럼 매핑이 자동이고 오타는 컴파일 에러.

HTTP 상태 코드 — 이번 주 쓴 것

코드	뜻	어디서
200 OK	성공	GET /books, POST /books(워크북 기본), PATCH 반납
201 Created	새 자원 생성	POST /rentals (@ResponseStatus(HttpStatus.CREATED))
404 Not Found	대상 없음	이미 반납된 rental 다시 반납 (ResponseStatusException)
500	서버 내부 오류	titel 오타, FK 999 — 4주차에 400으로 바꿀 것

미션 3개 · 핵심 키워드 · 셀프 체크

미션 1 필수 GET /books/category/{categoryId}

```
// Repository - WHERE category_id = ? 에 바인딩
public List<Map<String, Object>>
findByCategoryId(long categoryId) {
    String sql = "SELECT * FROM book WHERE
category_id = ? ORDER BY book_id";
    return jdbcTemplate.queryForList(sql,
categoryId);
}
// Controller - Path Variable
@GetMapping("/category/{categoryId}")
public List<Map<String, Object>>
getBooksByCategory(
    @PathVariable long categoryId) {
    return
bookService.getBooksByCategory(categoryId);
}
```

/1 → 3건, /2 → 1건, /999 → [] (빈 배열, 200)

미션 2 필수 POST /rentals

```
// Repository - NOW(), DATE_ADD + 생성된 rental_id 받
기
KeyHolder keyHolder = new GeneratedKeyHolder();
jdbcTemplate.update(connection -> {
    PreparedStatement ps =
connection.prepareStatement(
    sql, Statement.RETURN_GENERATED_KEYS);
    ps.setLong(1, userId); ps.setLong(2, bookId);
    return ps;
}, keyHolder);
return keyHolder.getKey().longValue();

// Controller - Body {"userId":1,"bookId":3} → 201
+ rentalId
@PostMapping @ResponseStatus(HttpStatus.CREATED)
public Map<String, Object>
createRental(@RequestBody Map<String, Object>
body) {
    long userId = ((Number)
body.get("userId")).longValue(); // JSON 숫자
=Integer
...
return Map.of("message", "대여 기록이 생성되었습니
다!", "rentalId", rentalId);
}
```

미션 3 선택 PATCH /rentals/{rentalId}/return

```
// Service - 갯수 0건이면 없거나 이미 반납 → 404 (규칙은
Service가)
if (rentalRepository.markReturned(rentalId) == 0)
    throw new
ResponseStatusException(HttpStatus.NOT_FOUND,
"...");
```

핵심 키워드 5 — 한 줄씩

다른 아 키텍처	계층형(이번 주) → 헥사고날(Port·Adapter로 의존 방향 통제) → 클린(동심원, 의존은 안쪽으로). MVC는 표현 계 층 패턴, MSA는 배포 단위 분리.
웹 보안 공격	SQL Injection(바인딩으로 방어) · XSS(이스케이프) · CSRF(토큰·SameSite) · 세션/비밀번호(bcrypt) · 민감정 보 노출(.env). OWASP Top 10.
커넥션 풀	비싼 DB 연결을 미리 N개 만들어 빌려주고 돌려받음. HikariCP 기본 10개, 첫 쿼리 때 초기화. 반납 안 하면 leak.
Raw SQL vs ORM	Raw는 쿼리 통제·튜닝 유리, 매핑 수동·타입 안전 없음. ORM은 객체↔테이블 자동, 학습 비용. 실무는 둘 다.
INSERT 외 SQL	DML(SELECT/UPDATE/DELETE) · DDL(CREATE/ALTER/DROP) · DCL(GRANT) · TCL(COMMIT/ROLLBACK → @Transactional).

셀프 체크

- ☐ INSERT에서 id 컬럼을 적지 않는 이유는?
AUTO_INCREMENT — DB가 번호를 붙인다.
- ☐ rental을 넣기 전에 반드시 있어야 하는 데이터는?
users와 book(그리고 book의 category). FK 부모가 먼저.
- ☐ Controller / Service / Repository 각각 한 단어로?
요청·응답 / 규칙 / SQL.
- ☐ 컬럼명이 바뀌면 어느 계층만 고치나?
Repository.
- ☐ 왜 매 요청마다 DB에 연결하지 않나?
연결 맺고 끊는 비용이 커서 서버가 뻘는다 → 풀에서 빌려 씀.
- ☐ queryForList와 update의 차이는?
조회(List<Map> 반환) vs 변경(영향 행 수 반환).
- ☐ "' + title + '" 가 위험한 이유와 대안은?
SQL Injection. ? 파라미터 바인딩.
- ☐ titel 오타가 400이 아니라 500인 이유는?
Map은 아무 Key나 받아서 검증 없이 DB까지 내려가 NOT
NULL에 걸린다. DTO+@Valid로 400.
- ☐ DB 비밀번호는 어디에 두나?
\${DB_PASSWORD:} 환경변수 + .env(gitignore). 코드·저장소
에 남기지 않는다.

연결해서 보기 — 같은 개념이 양쪽에 있다

개념	Flutter (go_router)	Spring Boot
Path Parameter	<code>GoRoute(path: '/movies/:movieId')</code> → <code>state.pathParameters['movieId']</code>	<code>@GetMapping("/category/{categoryId}")</code> → <code>@PathVariable long categoryId</code>
Query Parameter	<code>/movies?genre=SF</code> → <code>state.uri.queryParameters['genre']</code>	<code>@RequestParam String genre</code> (이번 주 미 사용, 같은 자리)
Body(객체) 전달	<code>extra: movie</code> — URL에 안 남음	<code>@RequestBody Map body</code> — JSON Body
없는 ID 처리	<code>findMovieById</code> → <code>null</code> → "찾을 수 없어요" 화면	<code>UPDATE ... WHERE rental_id=? 0건</code> → <code>404</code>
상태 소유자	선택 장르는 URL, Sheet 안 체크는 Sheet State, 평점은 부모 State	데이터는 DB, 규칙은 Service, 접속 정보는 환경변수
역할 분리	화면(Screen) / 조각 위젯(widgets/) / 라우팅(router/) / 데이터(data/)	Controller / Service / Repository
"나중에 바뀌어도 여기만"	새 화면 → <code>AppRouter.routes</code> 에만 추가	컬럼명 변경 → Repository SQL만 수정
8주차 · 4주차 예고	즐거찾기·평점을 실제 API로 (8주차)	DTO·ORM으로 오타·매핑·snake_case 문제 해결 (4주차)

지금 앱이 서버를 부른다면 (미래 그림)

```
Flutter                                Spring
context.push('/movies/1') → GET
/books/category/1

Service → Repository                  Controller →
SELECT * FROM
book WHERE category_id = ?
MovieDetailScreen ← JSON ← List<Map> → JSON
```

화면 쪽 Path Parameter가 그대로 서버 쪽 Path Variable이 된다.
응답 JSON의 `book_id` 같은 `snake_case`를 그대로 쓰면 앱까지 DB 스키마에 묶인다 — 그래서 DTO.

이번 주 판단 기록

- 상세 Route를 Shell 밖에 둬 → 전체 화면 + push/pop이 깔끔, `StatefulShellRoute` 불필요
- 장르 필터를 URL Query가 소유 → 목록 화면 Stateless, Challenge 충족
- POST /rentals는 201 + rentalId 반환 → 클라이언트가 바로 반납 API를 부를 수 있음
- 반납 UPDATE에 `AND returned_at IS NULL` → 반납일 덮어쓰기 방지, 규칙은 Service가 404로
- Initializr Gradle 오류 → Maven, `4.0.8.RELEASE` → `4.1.1` (Maven Central에서 직접 확인)